MODULE 29 · EXAM STRATEGY - ALL DOMAINS

Confusion Pairs

Forty-eight pairs across this course, sorted into the five patterns that generate almost all of them.

TYPE Synthesis MODULE 29 of 34 PAIRS 48 built

admin@sthithapragna.com

Most confusion pairs are the same few shapes wearing different service names. Learn the shapes, not the list.

The five patterns

PATTERN	THE QUESTION THAT RESOLVES IT
Grants against ceilings	Does this give permission, or only cap it?
Push against pull	Who initiates: the source, or the consumer?
Identifier against attribute	Do I need the name, or a property of the thing?
Capability against cost	Which feature does the scenario actually require?
Same word, two meanings	Which layer is this word being used at?

READ THIS TWICE

Five questions. Every pair in this course falls into at least one of them, and several fall into two.

Pattern one: grants against ceilings

- **Identity and resource policies grant.** They are the only two that do.
- **Permissions boundaries, SCPs and session policies cap.** None of them grants anything.
- **Reserved concurrency caps** a function's maximum. It also guarantees that much.
- **Provisioned concurrency allocates** warm environments. It is not a limit.
- The tell: if adding it alone changes nothing, it is a ceiling, not a grant.

> ON THE EXAM
>
> This pattern explains why 'we gave them administrator access and it still fails' is always a ceiling. Adding grants cannot defeat something that caps.

Pattern one, across the course

GRANTS	CAPS
Identity policy, resource policy	Permissions boundary, SCP, session policy
Provisioned concurrency	Reserved concurrency
Provisioned throughput on a table	Per-partition limits, which no mode removes
A KMS key policy statement	A grant constraint, or an encryption context condition
A usage plan's rate	Account-level API Gateway throttling, always applied

ON THE EXAM

Note the third row. Raising provisioned throughput never lifts a per-partition limit, which is why hot partitions throttle a table that looks under-used.

Pattern two: push against pull

- **Event source mappings poll.** Lambda reads from SQS, Kinesis and DynamoDB Streams.
- **Event notifications push.** S3, SNS and EventBridge invoke Lambda directly.
- That decides where retries live, and therefore what a DLQ is attached to.
- Polling means the consumer controls the rate. Pushing means the source does.
- Same shape: SQS consumers pull, SNS subscribers are pushed to.

> ON THE EXAM
>
> This pattern determines whether a failure is retried by Lambda or by the source, which is why destinations work for async and not for event source mappings.

Pattern two, across the course

PULL	PUSH
SQS, Kinesis, DynamoDB Streams	S3 events, SNS, EventBridge
Event source mapping, with a batch	Asynchronous invocation, one event
Retries handled by the mapping	Retries handled by Lambda, twice, then a DLQ
maxReceiveCount and a DLQ on the queue	Destinations, on success or on failure
The consumer sets the pace	The source sets the pace, so throttling matters

ON THE EXAM

The fourth row is the practical one. Destinations require asynchronous invocation, which is why they silently do nothing on an SQS-triggered function.

Pattern three: identifier against attribute

- **Ref returns the physical ID. GetAtt returns a named attribute.**
- **An annotation is indexed** and searchable. **Metadata is stored** and is not.
- An IAM role's Ref gives its **name**. Most places need its **ARN**, which is GetAtt.
- An S3 **bucket ARN** and its **object ARN** are different strings. Listing needs one, reading the other.
- The tell: if the scenario needs to **find** or **reference** something, ask which string it needs.

> ON THE EXAM
>
> Every pair here fails the same way: the value is present, has the right shape, and is the wrong string. The error usually names what it expected.

Pattern three, in reverse

- Not every case wants the attribute. Sometimes the identifier is correct.
- A **bucket ARN** is right for ListBucket. The **object ARN** is right for GetObject.
- A **queue URL** is what SendMessage takes. The **queue ARN** is what a policy names.
- An **alias ARN** is right for a client. A **version ARN** is right for a deliberate pin.
- So the question is never which is better, only which one this operation expects.

> ON THE EXAM
>
> The pattern is not 'always use GetAtt'. It is 'know which string the operation takes', and the documentation states it for every action.

Pattern three, across the course

THE IDENTIFIER	THE ATTRIBUTE YOU USUALLY WANT
Ref on an IAM role gives the name	GetAtt Arn gives the ARN
Ref on an RDS instance gives an ID	GetAtt Endpoint.Address gives the hostname
Ref on an SQS queue gives the URL	GetAtt Arn gives the ARN
Ref on an SNS topic gives the ARN	Already the ARN. The inconsistency the exam likes
X-Ray metadata is stored	An annotation is indexed and filterable
A version ARN never moves	An alias ARN follows your deployments

ON THE EXAM

The SNS and SQS rows together are a favourite: Ref gives a topic ARN but a queue URL. There is no principle behind it, so it has to be remembered.

Pattern four: capability against cost

- **REST or HTTP API** turns on six features, not on preference.
- **Secrets Manager or Parameter Store** turns on rotation, sharing and replication.
- **Standard or Express** turns on five minutes and exactly-once.
- **Provisioned or on-demand** turns on predictability.
- In each pair the cheaper option is correct **unless** a named capability forces the other.

> ON THE EXAM
>
> The reasoning direction matters. Do not ask which is better; ask whether the scenario names a capability only one of them has. If not, the cheaper one wins.

Pattern four: what forces the dearer option

PAIR	THE CAPABILITY THAT DECIDES
REST or HTTP API	API keys, usage plans, caching, validation, mapping templates, WAF
Secrets Manager or Parameter Store	Automatic rotation, cross-account, cross-Region
Standard or Express workflow	Over 5 minutes, exactly-once, or the callback pattern
Provisioned or on-demand capacity	Predictable traffic makes provisioned cheaper
DAX or ElastiCache	DAX only fronts DynamoDB, with no code change
KMS or CloudHSM	Single-tenant hardware, or a PKCS#11 interface

ON THE EXAM

Memorising the middle column is worth more than memorising the pairs. The capability is what a question names; the pair is only the two options offered.

Pattern five: same word, two meanings

- **Canary** is a traffic-shifting deployment, and also a CloudWatch Synthetics script.
- **Version** is a Lambda snapshot, an S3 object version, and a CloudFormation template version.
- **Alias** points at a Lambda version, and also names a KMS key.
- **Grant** is a KMS delegation, and also an ordinary English verb in policy documents.
- **Execution role** is Lambda's identity, and an **ECS task execution role** is something else entirely.

> ON THE EXAM
>
> These are the ones that catch people who know the material. The word is familiar, the meaning is from a different module, and the answer looks obviously right.

The near-synonyms

- **Error**, **fault** and **throttle** in X-Ray mean 4xx, 5xx and 429 specifically.
- **Authentication** proves identity. **Authorisation** decides what is permitted.
- **Reserved** and **provisioned** are opposites in Lambda and near-synonyms elsewhere.
- **Quota**, **limit** and **throttle**: the first two are the ceiling, the third is hitting it.
- Precision here is cheap to acquire and reliably worth a mark or two.

> ON THE EXAM
>
> The reserved and provisioned pair is the nastiest, because the words behave one way in Lambda and the opposite way in DynamoDB and EC2.

Pattern five, in full

WORD	THE TWO MEANINGS
Canary	Traffic-shifting deployment, or a Synthetics availability script
Execution role	Lambda's own identity, or ECS's role for starting a container
Alias	A movable pointer to a Lambda version, or a friendly name for a KMS key
Endpoint	A service URL, a VPC endpoint, or an RDS hostname
Policy	IAM policy, bucket policy, key policy, stack policy, scaling policy
Destination	Lambda's async destination, or a CloudWatch Logs cross-account destination
Stage	An API Gateway stage, or a CodePipeline stage

ON THE EXAM

When a familiar word appears in an unfamiliar context, slow down. The meaning is set by the service, not by what you learned first.

The pairs that blur between modules

THESE LOOK SIMILAR	BECAUSE THEY ARE AT DIFFERENT LAYERS
API Gateway canary and Lambda alias weights	The gateway shifts between deployments; the alias between versions
Beanstalk immutable and CodeDeploy blue/green	Both replace rather than update. Different services, same idea
CloudFormation change set and cdk diff	Both preview. One is the service, one is the tool
Step Functions Retry and SDK retries	One is workflow configuration, one is inside the API client
SQS DLQ and Lambda DLQ	One belongs to the queue, one to the function
Cognito authoriser and Lambda authoriser	One validates a token; one runs your code

ON THE EXAM

Every row is two correct mechanisms operating at different layers. A question naming one layer is never answered by the other.

The pairs decided by one number

- **15 minutes:** Lambda's ceiling. Past it, Fargate, ECS or Step Functions.
- **29 seconds:** API Gateway's integration timeout, and the reason for 504.
- **5 GB:** the S3 single-PUT limit, above which multipart is mandatory.
- **4 KB:** the KMS Encrypt limit, and why envelope encryption exists.
- **400 KB:** a DynamoDB item, and why large objects go to S3 with a pointer.

> ON THE EXAM
>
> Five numbers that each decide an entire class of question. If a scenario states a size or a duration, check it against this list first.

The numbers that separate a pair

NUMBER	WHAT IT SEPARATES
256 KB	A Step Functions state payload, and an SQS or SNS message
6 MB	A Lambda synchronous response, and its asynchronous payload
10 MB	An API Gateway payload, above which streaming or S3 is needed
1 MB	A DynamoDB Query or Scan page, and a Kinesis record batch
300 seconds	API Gateway's raised timeout ceiling, against the 29-second default
12 hours	An SQS visibility timeout maximum, and an STS session maximum

ON THE EXAM

Several of these numbers appear in more than one service, which is exactly why a question can state a size and expect you to know which limit it is testing.

The pairs decided by direction

- **Identity policy:** what my principals may do. **Resource policy:** who may touch my resource.
- **Trust policy:** who may assume this role. **Permissions policy:** what the role may then do.
- **X-Ray:** requests flowing through your services. **CloudTrail:** API calls changing your account.
- **ID token:** who the user is. **Access token:** what they may do.
- The tell: ask whether the scenario is about the caller or about the thing being called.

> ON THE EXAM
>
> Direction pairs are the easiest to resolve once noticed and the easiest to answer wrongly at speed, because both options are real and both sound relevant.

ALL DOMAINS / AND A CLAUSE DECIDES

The pairs where both are right

- Several pairs have **no wrong answer in general**, only a wrong answer for this scenario.
- RDS Proxy and the Data API both solve Lambda's connection problem, differently.
- SAM and the CDK both produce CloudFormation. So does CloudFormation.
- Redis and Memcached both cache. Fargate and EC2 both run containers.
- In these, **find the disqualifying clause** rather than comparing the options.

> ON THE EXAM
>
> Modules 10 and 14 built this habit deliberately. If you cannot name a clause that eliminates your second choice, you have not found the real constraint yet.

The pairs most often got wrong

PAIR	THE SINGLE FACT THAT RESOLVES IT
Task role or task execution role	Failure before the container starts is the execution role
Ref or GetAtt	Need an ARN or an endpoint? GetAtt
Metric filter or subscription filter	A number, or the events themselves?
Annotations or metadata	Only annotations are indexed
User pool or identity pool	Calling an AWS service directly needs both
Reserved or provisioned concurrency	Reserved is a limit and costs nothing
502 or 504	Answered wrongly, or did not answer in time

ON THE EXAM

Worth screenshotting. Seven pairs, seven single facts, and between them a meaningful share of the marks the exam puts on this kind of distinction.

How to use a pair under pressure

- **Name the pattern first.** Grant or ceiling, push or pull, identifier or attribute?
- **Then find the clause** in the scenario that only one option satisfies.
- If both still fit, you have missed a clause. Reread rather than guessing.
- Check the qualifier: most cost-effective, least operational overhead, minimal code change.
- Eliminate options belonging to a different **layer** or a different **platform**.

> ON THE EXAM
>
> That last step is fast and reliable. ValidateService in a Lambda question, or a queue URL where an ARN is needed, are eliminations you can make without deep thought.

Four scenarios, each built on a pair. Name the pattern before you name the answer.

admin@sthithapragna.com

> A media company distributes large video files to paying subscribers worldwide, and separately lets those subscribers upload their own footage. Both must be restricted to authenticated users, and the bucket must stay private.
>
> Downloads must remain available for 30 days after purchase. Uploads happen immediately after the user selects a file.

What should be used for each direction?

A. CloudFront signed URLs for downloads, and S3 presigned URLs for uploads

B. S3 presigned URLs for both directions

C. CloudFront signed URLs for both directions

D. A public bucket with a bucket policy restricting access by referer header

◆ PAUSE AND DECIDE

What should be used for each direction?

A CloudFront down, presigned up

Downloads need 30 days and global distribution, which only CloudFront gives. Uploads must be presigned, because CloudFront cannot sign an upload at all.

B Presigned for both

Presigned URLs cap at 7 days with IAM user credentials, and far less from a role. 30 days is impossible, and there is no edge caching.

C CloudFront for both

CloudFront signed URLs are download only. There is no upload equivalent, so this cannot be built as described.

D Public bucket, referer check

Contradicts 'the bucket must stay private', and a referer header is trivially forged. Making a bucket public is essentially always wrong.

THE SIGNAL

A capability pair with a direction twist. The two requirements point at different services, and the giveaway is that only one of the pair can sign an upload at all.

> An account runs 40 Lambda functions sharing a 1,000 concurrent execution limit. A batch reporting function occasionally scales to hundreds of concurrent executions and starves the customer-facing checkout function, which then throttles.
>
> The checkout function must always have capacity available, and the batch job may run slower.

What should be configured?

A Provisioned concurrency on the checkout function

B Reserved concurrency on the batch function, capping it well below the account limit

C Request an account concurrency increase and change nothing else

D Provisioned concurrency on the batch function so it runs faster and finishes sooner

◆ PAUSE AND DECIDE

What should be configured?

A. Provisioned concurrency on checkout

The near miss. It reserves warm environments for checkout and costs money continuously, and it does not stop the batch job consuming the shared pool.

B. Reserved concurrency on the batch function

Reserved concurrency caps the batch job, so it can never consume more than its share. It costs nothing, and the batch simply runs slower.

C. Raise the account limit

Buys headroom until the batch job scales further. It does not prevent one function starving another, which is the actual problem.

D. Provisioned concurrency on the batch

Makes the batch function start faster, which makes it consume the shared pool sooner. It worsens the problem.

THE SIGNAL

A grant-or-ceiling pair. The requirement is to cap the noisy neighbour, and only reserved concurrency caps. Provisioned concurrency allocates, which is the opposite.

> A compliance team must retain every application log line for seven years in a searchable archive outside CloudWatch. Separately, the operations team wants a page when the phrase FATAL appears more than five times in ten minutes.
>
> Both requirements apply to the same log group.

What should be configured?

A. Two metric filters, one counting FATAL and one counting all lines

B. A subscription filter for both, with a Lambda function that counts and alerts

C. A subscription filter to Firehose for the archive, and a metric filter with an alarm for FATAL

D. Set retention to seven years and create a CloudWatch alarm on the log group

◆ PAUSE AND DECIDE

What should be configured?

A Two metric filters

Metric filters produce counts. A count cannot be archived and searched, so the compliance requirement is unmet.

B Subscription filter for both

Delivers the events, then rebuilds counting and alerting in a function that a metric filter and alarm already do for nothing.

C Subscription for archive, metric filter for the alarm

Two requirements, two mechanisms. The subscription filter delivers events for archival; the metric filter produces the number the alarm needs.

D Retention plus a log group alarm

You cannot alarm on a log group directly, only on a metric derived from it. Seven years in CloudWatch is also expensive and not outside CloudWatch.

THE SIGNAL

The pair does not resolve to one option. Two requirements need two mechanisms, and recognising that is what the question tests.

A mobile app signs users in with a Cognito user pool. Its backend API, behind API Gateway with a Cognito authoriser, needs to know the caller's identity and their group membership. The developer currently sends the ID token in the Authorization header.

A security review objects that user attributes including email and date of birth are being transmitted to a service that does not need them.

What should change?

A Move to an identity pool, so the API receives AWS credentials instead

B Remove the attributes from the user pool schema

C Encrypt the ID token before sending it

D Send the access token instead, which carries scopes and cognito:groups but not user attributes

◆ PAUSE AND DECIDE

What should change?

A Move to an identity pool

Identity pools issue AWS credentials for calling AWS services directly. The app calls its own API, so a user pool alone is correct.

B Remove the attributes

The application genuinely needs email and date of birth. Deleting them from the pool solves the transmission by destroying the data.

C Encrypt the token

It already travels over TLS, and the objection is that the API receives the attributes at all, not that they are intercepted.

D Send the access token

The access token carries scopes and cognito:groups, which is exactly what the API needs, without the user attributes it does not.

THE SIGNAL

A direction pair. The ID token says who the user is, for your application; the access token says what they may do, for an API. Sending the wrong one is the classic mistake.

What to carry forward

- **Five patterns generate almost every pair.** Name the pattern before the answer.
- **Grants against ceilings** explains why more permissions sometimes change nothing.
- **Push against pull** decides where retries live and what a DLQ attaches to.
- For capability pairs, the cheaper option wins unless a named capability forces the other.
- Some scenarios use **both** halves of a pair. Two requirements can need two mechanisms.

> ON THE EXAM
>
> Next: how DVA-C02 questions are written, which turns these habits into a reading technique for the exam itself.